Beating Hallucination in Sutra — Research, Tools & a Plan

> Deep-dig on LLM hallucination: what causes it, every mitigation family, the real open-source tools (GitHub stars noted), and a concrete plan mapped onto Sutra's stack. *Compiled 2026-10-07 from recent surveys (2025–2026), primary papers, and GitHub.*

---

0. TL;DR — the honest framing

Sutra's problem isn't "the model lies sometimes." It's **three specific failure modes**, and each has a *different* fix:

1. **Phantom tool successes / phantom tool calls** — the agent claims it did something it didn't, or calls a tool/args that don't exist. *This is your #1.* → **Deterministic, $0 fixes** (registry validation, schema-checked dispatch, verify-the-result-before-claiming). No model needed.
2. **Factual / faithfulness hallucination** — makes up facts not grounded in retrieved context or tool output. → **Grounding + a local verifier model + abstention.**
3. **Long-chain drift** — weak/local models lose the plot at 3+ steps (your observed wall). → **Decompose + verify each step + refresh context + escalate the hard synthesis step.**

The literature is unanimous on one thing: **no single method fixes it — you layer complementary defenses.** The good news: the cheapest, most deterministic layers kill the *agentic* failures that hurt you most, and they cost **$0** and need no cloud.

> Reality check from the field: *Gartner predicts >40% of agentic-AI projects get dropped by 2027 over reliability.* Nailing this **is** the moat.

---

1. Why it happens (causes)

- **Next-token prediction has no truth signal** — fluency ≠ factuality; the model optimizes plausibility.
- **Weaker/smaller models hallucinate more** — exactly your local-tier wall (local fails 1 synthesis; free cloud drops at 3+ steps; Claude holds). Capability is the dominant factor.
- **Big tool inventories** — more tools → harder to tell a real tool from an imagined one (phantom calls rise with inventory size). *(You already fight this with on-demand* `select_tools`*.)*
- **Long / multi-step workflows** — tool-schema awareness degrades across a long context; the model "forgets" what's real.
- **Documentation gaps** — ambiguous tool specs → the model invents.
- **The RL-reasoning paradox** (ICLR 2026): training a model for *stronger reasoning* can **raise** its hallucinated-tool-call rate in lockstep with accuracy. More "reasoning" ≠ fewer phantoms.

---

2. The mitigation toolkit (by family)

A. Agentic / tool-call defenses — *cheapest, highest ROI for Sutra, all deterministic & $0*

From the phantom-tool-call research, layer these in the **agent loop** (`core/agent.py`): 1. **Tool-registry validation** — before dispatch, check the tool name is in the registry. Reject unknowns. (You have the registry; add the guard.) 2. **Schema-validated dispatch** — validate args with Pydantic/JSON-Schema before running. *Reported: parameter errors 40% → 2%.* 3. **Unknown-tool circuit breaker** — on a bad call, feed back *"that function doesn't exist; available: [list]"* so the model self-corrects instead of spiraling. 4. **Pre-execution planning** — have the agent outline the whole tool sequence (placeholders) and validate it against the registry *before* running anything. *Reported: 60% → 85% accuracy.* (Your `planner` already decomposes — add a registry check to each step.) 5. **Dynamic tool retrieval** — expose only the 5–10 relevant tools per step. **You already do this** (`select_tools` , `SUTRA_MAX_TOOLS`) — the research validates it. 6. **Context refresh** — re-inject tool schemas + task state at each decision point, not just once — fixes long-chain "tool-awareness decay." 7. **Verify-before-claim (the phantom-success killer)** — after a tool runs, check the *actual* result/postcondition deterministically before the model is allowed to say "done." This is the code version of your "no phantom successes" rule.

B. Grounding & RAG — *most reliable single factual strategy*

- **Strict citation contract**: every factual claim must cite a retrieved passage; **abstain** if no passage supports it. (You have RAG + graph memory — enforce the contract + abstention.)
- **Knowledge-graph grounding** gives traceable answers (your FalkorDB graph is an asset here).
- Measure with **faithfulness/groundedness** metrics (see §D).

C. Constrained / structured decoding — *eliminates a whole error class by construction*

Force output to a grammar/JSON-Schema by masking invalid next-tokens. *Research: 65% of schema errors are "hallucinated keys" + malformed brackets — constrained decoding removes both entirely.* Perfect for **tool-call args** and any structured output. - **Outlines** (`dottxt-ai/outlines` , 15.9k★) — FSM-based constrained decoding, plugs into HF Transformers as a logit processor → works with your **local Ollama/HF models**. - **Instructor** (`567-labs/instructor` , 14k★) — Pydantic-validated outputs + auto-retry (wraps APIs). - **Guidance** (Microsoft) — logit-processor constraints, HF-integrated. - ⚠️ *Alignment-tax caveat*: over-constraining can hurt free-form reasoning/reflection — constrain the **structured** parts (args, JSON), not the thinking.

D. Self-verification loops

- **Chain-of-Verification (CoVe)** (arXiv 2309.11495): draft → generate verification questions → answer them **independently** (don't let it peek at the draft) → final verified answer. Big win on your weak-synthesis step; prompt-only, no finetune.
- **Self-Refine** / **Reflexion**: iterative generate→critique→revise with one model, prompt-only.

E. Uncertainty, abstention & the "I don't know" filter — *the truth-discipline layer*

- **SelfCheckGPT** (`potsawee/selfcheckgpt` , 634★): zero-resource, black-box. Sample N responses at temp 1; if they disagree (NLI contradiction / BERTScore), it's likely hallucinated. **No external DB, works on any local model** — $0.
- **Semantic entropy** (*Nature* 2024): cluster samples by *meaning* (entailment), measure entropy = meaning-level uncertainty. More principled than token entropy.
- **"I Don't Know" filter for function-calling** (arXiv 2607.04034): a lightweight trainable uncertainty filter that suppresses risky tool calls and lets the agent *abstain*. Directly on-point.

- **Policy**: calibrated low-confidence → **abstain or escalate**, never fabricate. (This *is* your reliability-safe arch: tools find / model reports / human submits.)

F. Detection models as local verifiers — *run these as a $0 second opinion*

Small, open models that judge "is this claim supported by this context?" — plug in after retrieval / before surfacing an answer: - **Bespoke-MiniCheck-7B** (`bespokelabs`) — SOTA fact-grounding; **available on Ollama** (`ollama pull bespoke-minicheck`) → local, fast, $0. (MiniCheck-FT5 is only 770M and hits GPT-4-level grounding at ~400× lower cost — arXiv 2404.10774.) - **Patronus Lynx 8B** (HuggingFace) — SOTA open hallucination detector for RAG (checks: supported / no-extra-info / no-contradiction). Integrates with NeMo Guardrails. 8B fits your 16GB GPU. - **Vectara HHEM-2.1-Open** (HuggingFace) — tiny, fast hallucination-evaluation model + a public leaderboard of model hallucination rates.

G. Guardrail frameworks — *the orchestration layer*

- **Guardrails AI** (`guardrails-ai/guardrails` , 7.5k★) — validators (incl. hallucination), re-ask, structured output.
- **NeMo Guardrails** (`NVIDIA-NeMo/Guardrails` , 7.3k★) — programmable rails incl. a fact-checking / Lynx hallucination rail. Heavier; good for a final gate.

H. Measurement / eval — *you can't fix what you don't measure (wire into CI)*

- **DeepEval** (`confident-ai/deepeval` , 18.7k★) — pytest-style LLM eval (faithfulness, hallucination metrics) → drops straight into your existing CI.
- **RAGAS** (`vibrantlabsai/ragas` , 15.9k★) — RAG faithfulness/answer-relevancy/context metrics.
- **TruLens** (`truera/trulens` , 3.6k★) — eval + tracing for LLM **agents**.
- **UpTrain** (2.4k★); **Vectara hallucination-leaderboard** (3.3k★) — pick a known-low-hallucination model for critical tiers.

---

3. Consolidated tool table

Tool	Stars	Category	Local/$0?	Fit for Sutra
Outlines	15.9k	constrained decoding	✅ (HF/Ollama)	Tool-arg + JSON grammar enforcement

Tool	Stars	Category	Local/$0?	Fit for Sutra
Instructor	14k	structured output + retry	⚠️ API-wrapper	Validated outputs on cloud tiers
DeepEval	18.7k	eval	✅	Hallucination/faithfulness tests in CI
RAGAS	15.9k	RAG eval	✅	Grounding/faithfulness scoring
Guardrails AI	7.5k	guardrail validators	✅	Output validation + re-ask
NeMo Guardrails	7.3k	rails + fact-check	✅/mixed	Final fact-check gate (Lynx rail)
TruLens	3.6k	agent eval/tracing	✅	Trace where hallucinations enter
Vectara leaderboard/HHEM	3.3k	detection model + ranking	✅ (HF)	Tiny local groundedness check
SelfCheckGPT	634	zero-resource detection	✅	$0 consistency check, any model
Bespoke-MiniCheck-7B	— (Ollama)	grounding verifier	✅ (Ollama)	**Local $0 claim-vs-context check**
Patronus Lynx 8B	— (HF)	RAG hallucination detector	✅ (16GB GPU)	Local RAG faithfulness judge

---

4. Recommended plan for Sutra (prioritized)

Tier 1 — deterministic, $0, do first (kills the phantom failures that hurt most): 1. **Agent-loop guards** in `core/agent.py`: tool-registry validation → schema-validated dispatch (Pydantic) → unknown-tool circuit breaker. *Biggest reliability win per line of code.* 2. **Verify-before-claim**: for every acting skill, assert the real postcondition before reporting success (file exists, exit 0, element present…). Turns your "no phantom successes" rule into enforced code. Add to the acted-signal/stall-guard you already have. 3. **Constrained decoding for tool args** via **Outlines** on local models → no malformed/invented args. 4. **Context refresh** in the planner: re-inject the tool schemas + current step state each step.

Tier 2 — local verifier + abstention ($0, one model pull): 5. `ollama pull bespoke-minicheck` → a **grounding gate**: before surfacing a factual answer, check each claim is supported by retrieved context / tool output; unsupported → abstain or re-retrieve. 6. **Abstention policy**: wire an "I don't know / escalate" path on low confidence (your tiered router can escalate the *hard* step to Claude instead of letting a local model fabricate). 7. **SelfCheckGPT-style self-consistency** on high-stakes factual turns only (sample N, check agreement) — gate it so it doesn't tax every turn.

Tier 3 — reasoning upgrades + measurement: 8. **CoVe** on synthesis steps (your weak spot at 3+ steps) — draft → verify-questions → final. 9. **DeepEval + RAGAS hallucination/faithfulness suite in CI** → a regression gate so reliability can't silently rot (you already have strong CI — this fits). 10. Pick critical-tier models by the **Vectara leaderboard** (lowest hallucination rate per $).

What you already got right (keep): on-demand tool selection, the confirmation gate, planner decomposition + save/resume, the free-worker $0 guarantee, and the tools-find/model-reports/human-submits architecture. The research *validates* all of these.

---

5. Key sources

- **Surveys:** Comprehensive LLM Hallucination Survey (arXiv 2510.06265) · Application-Oriented Survey on RAG/Reasoning/**Agentic** (arXiv 2510.24476) · Lifecycle-Based Survey (arXiv 2608.26168).
- **Agentic:** "Phantom Tool Calls" (tianpan.co, 2026-04) · "I Don't Know Filter for Function Calling" (arXiv 2607.04034).
- **Techniques:** Chain-of-Verification (arXiv 2309.11495) · SelfCheckGPT (arXiv 2303.08896) · Semantic Entropy (*Nature*, PMC11186750) · Constrained-decoding alignment tax (arXiv 2604.06066).
- **Detectors:** MiniCheck (arXiv 2404.10774) · Patronus Lynx (patronus.ai) · Vectara HHEM-2.1-Open.
- **Tools:** Outlines · Instructor · DeepEval · RAGAS · Guardrails AI · NeMo Guardrails · TruLens · SelfCheckGPT · Bespoke-MiniCheck (Ollama).

Next step if you want it: I can prototype the Tier-1 agent-loop guards + a Bespoke-MiniCheck grounding gate against your actual `core/agent.py` and `skills/registry.py`.